Informe del Laboratorio 02

Decisiones, ciclos y colecciones en Python - Gestor de inventario

Aprendiz	Anthony Córdova Calderón
Ficha	3388087
Programa	ADSO
Laboratorio	Laboratorio 02 de Python: decisiones, ciclos, listas, tuplas, conjuntos, diccionarios, comprensiones y actividad integradora (Gestor de inventario)
Fecha del informe	21 de octubre de 2026
Repositorio de GitHub	https://github.com/skyyleerr/python-laboratorios
Carpeta del laboratorio	laboratorio-02/ (el laboratorio-01/ se conserva sin cambios)

1. Descripción del gestor de inventario y de las funcionalidades

`gestor_inventario.py` es una aplicación de consola que administra productos en memoria (sin archivos ni bases de datos). Muestra un menú que se repite hasta elegir **0**:

Opción	Funcionalidad	Reglas aplicadas
1. Registrar	Pide código, nombre, categoría, precio y cantidad y agrega un diccionario a la lista.	Código no repetido; campos de texto no vacíos (ni solo espacios); categoría dentro de la tupla CATEGORIAS; precio mayor que cero; cantidad no negativa (cero se acepta como producto sin stock).
2. Listar	Muestra código, nombre, categoría, precio y unidades de cada producto.	Si la lista está vacía informa que no hay registros.
3. Buscar por código	Recorre la lista y muestra el producto encontrado.	Si no existe, informa al usuario. El código se normaliza con strip().upper().
4. Actualizar existencias	Reemplaza la cantidad del producto encontrado.	Rechaza negativos y conserva la cantidad anterior; los demás campos no cambian.
5. Eliminar	Quita el diccionario de la lista.	Si el código no existe muestra un mensaje, sin error.
6. Sin stock	Lista los nombres con cantidad cero (comprensión de lista).	-
7. Resumen	Productos, unidades, valor del inventario (precio * cantidad) y categorías usadas (comprensión de conjunto).	Usa siempre los datos actuales.

Además se desarrollaron los ejercicios de las prácticas 2.1 a 2.8, el reto de ampliación (`gestor_inventario_ampliado.py`: aumentar o disminuir existencias, orden por nombre, diccionario código-nombre, búsqueda por categoría y producto de mayor valor), el reto de depuración (`gestor_depuracion.py`) y las pruebas automáticas (`pruebas_gestor.py`).

2. Estructuras de control y colecciones utilizadas

Elemento	Dónde se usa y por qué
while	Ciclo del menú: se repite hasta que la opción sea "0".
if / elif / else	Selecciona la acción del menú y valida cada dato antes de modificar la información.
and / or / not y comparaciones	Reglas como precio <= 0 o cantidad < 0; en 2.2, acceso a una sala o evaluación de solicitudes.
for y break	Recorre la lista para buscar un código; break detiene el ciclo cuando ya se encontró.
Lista	**productos**: guarda todos los registros y permite agregar (append) y eliminar (remove).

Elemento	Dónde se usa y por qué
Diccionario	Cada producto: claves codigo, nombre, categoria, precio y cantidad; permite actualizar un solo campo.
Tupla	**CATEGORIAS**: opciones fijas que no deben cambiar durante la ejecución.
Conjunto	Categorías utilizadas: guarda cada valor una sola vez.
Comprensiones	De lista (productos sin stock), de conjunto (categorías) y sum() sobre una comprensión (valor del inventario); en 2.8 también de diccionario.

Se eligió cada estructura según la necesidad: lista para registros que cambian, diccionario para datos con nombre, tupla para datos fijos y conjunto para valores únicos.

3. Estado del proyecto

Avances

- Prácticas 2.1 a 2.8 con todos sus ejercicios, retos y depuraciones.
- Gestor de inventario completo, con las siete opciones y todas las validaciones; pasa 19 de 19 pruebas automáticas.
- Reto de ampliación y reto de depuración (cinco errores explicados).
- README del laboratorio, README principal con los laboratorios 01 y 02, y este informe.

Funcionalidades pendientes

- Ninguna funcionalidad obligatoria. Quedan fuera de este laboratorio, por indicación del enunciado: el manejo de excepciones (escribir `abc` donde se pide un número), las funciones propias y la persistencia en archivos.

Dificultades y soluciones

Dificultad	Solución
El listado y el resumen no coincidían con el formato del enunciado ("$ 85000.0" con espacio y el conjunto de categorías impreso con llaves y en orden variable).	Se pasó a f-strings (`$85000.0`) y se muestran las categorías ordenadas con sorted() y join() (commit cedb766).
En una ejecución manual los precios se escribieron con punto de miles (85.000, 45.000, 700.000). Python lo lee como 85.0, y el resumen mostró $1740.0 en lugar de $1740000.0.	No era un error de cálculo sino del dato escrito. Se documentó en el README escribir los precios sin puntos (85000) y las pruebas automáticas usan ese formato.
El caso de prueba de código repetido fallaba (código de salida 1).	El gestor detecta el duplicado antes de pedir precio y cantidad; la prueba enviaba líneas de más. Se corrigió la prueba (commit e4667f3).
acceso_actividad.py y validar_compra.py estaban vacíos y faltaban ejercicios de listas (duplicados, operaciones, cortes, copias, lista de tareas).	Se completaron en los commits 6ab5054 y 774b477.
La explicación de ciclo_queno_termina_co.py estaba incompleta.	Se completó (commit 85ed6a4).

4. Seguimiento de commits relevantes

Historial real y verificable con `git log --oneline`. El laboratorio 01 se conserva intacto en su carpeta.

Commit	Mensaje	Qué se modificó	Por qué	Resultado
23bb936	chore(lab2): crear carpeta laboratorio-02	Se creó la carpeta y se ignoró .vscode.	Separar el laboratorio 02 sin tocar el 01.	Estructura lista.

Commit	Mensaje	Qué se modificó	Por qué	Resultado
00b64bd	feat(2.1): comparaciones y decisiones	Se agregaron 9 archivos de if/elif/else.	Practicar condiciones y la calculadora de envío.	Casos límite 99999, 100000, 199999, 200000 verificados.
d079f16	feat(2.2): condiciones compuestas	Se agregaron 9 archivos con and/or/not y rangos.	Combinar condiciones y validar.	Reglas de acceso y rangos funcionando.
fc5fd24	feat(2.3): ciclos for y while	Se agregaron 11 archivos de ciclos.	Contadores, acumuladores y menú repetitivo.	Ciclos que terminan correctamente.
1955fa8 a bc849cd	feat(2.4) a feat(2.8)	Listas, tuplas, conjuntos, diccionarios y comprensiones (5 commits, uno por práctica).	Estudiar cada colección por separado antes del reto.	Base para el gestor.
6ab5054	feat(2.2): completar archivos vacíos	acceso_actividad.py y validar_compra.py.	Estaban sin contenido.	Cuatro combinaciones de and y cuatro casos de compra verificados.
774b477	feat(2.4): completar ejercicios de listas	Se agregaron 8 archivos faltantes.	Cubrir duplicados, operaciones, cortes, copias y lista de tareas.	Todos se ejecutan correctamente.
85ed6a4	fix(2.3): explicación incompleta	Se completó el comentario.	Documentar por qué termina el ciclo.	Texto completo.
5920aaa	feat(2.9): gestor_inventario.py	Primera versión del gestor con las 7 opciones.	Implementar el reto integrador.	Funcional; formato de salida con diferencias.
b457314	test(2.9): pruebas automáticas	Se agregó pruebas_gestor.py con 19 pruebas.	Verificar las pruebas obligatorias sin repetirlas a mano.	16 de 19 correctas.
e4667f3	test(2.9): corregir caso de código repetido	Se ajustó la entrada de la prueba.	El duplicado se detecta antes de pedir el precio.	17 de 19.
cedb766	fix(2.9): formato del enunciado	f-strings en listado, búsqueda y resumen; categorías ordenadas.	Coincidir con el ejemplo del laboratorio.	19 de 19 correctas.
dc8e88c	feat(2.9): reto de ampliación	gestor_inventario_ampliado.py con opciones 8 a 12.	Movimientos sin negativos, orden, diccionario, categoría y mayor valor.	Cada opción verificada.
f5e092f	feat(2.9): reto de depuración	gestor_depuracion.py con cinco errores comentados.	Explicar causa y corrección.	Total incorrecto 830000.0 vs. correcto 1740000.0.
f948708	docs(lab2): README	README del laboratorio 02 y README principal.	Describir ejecución de los dos laboratorios.	Instrucciones completas.
(último)	docs: agregar informe PDF	informe/Informe_Laboratorio_02.pdf.	Entregar el informe con el código.	Repositorio completo.

5. Evidencias de las pruebas

`python pruebas_gestor.py` ejecuta el gestor enviando opciones por la entrada estándar y comprueba cada resultado esperado.

Área	Prueba	Comportamiento esperado	Resultado
Registro	Listar sin productos	Mensaje de inventario vacío	Correcto
Registro	Registrar P001	Se agrega	Correcto
Registro	Registrar P001 de nuevo	Rechaza código repetido	Correcto
Registro	Nombre vacío / solo espacios	Rechaza	Correcto
Registro	Categoría fuera de la tupla	Rechaza	Correcto

Área	Prueba	Comportamiento esperado	Resultado
Registro	Precio negativo	Rechaza	Correcto
Registro	Cantidad negativa	Rechaza	Correcto
Registro	Cantidad cero	Acepta como producto sin stock	Correcto
Búsqueda	Buscar P001	Muestra su registro	Correcto
Búsqueda	Código desconocido	Informa que no existe	Correcto
Actualización	Actualizar a 10 unidades	Cambia solo la cantidad	Correcto
Actualización	Actualizar a -3	Conserva la cantidad anterior (4)	Correcto
Eliminación	Eliminar existente	Borra el registro	Correcto
Eliminación	Eliminar inexistente	No falla	Correcto
Reportes	Resumen con P001 y P002	2 productos, 4 unidades, $340000.0, Periféricos	Correcto
Reportes	Comprobación numérica P001-P003	3 productos, 6 unidades, $1740000.0, 2 categorías, Mouse sin stock	Correcto
General	Opción no válida / Salir	Mensaje / termina el ciclo	Correcto

Cálculo manual de la comprobación numérica: 85000 x 4 = 340000; 45000 x 0 = 0; 700000 x 2 = 1400000; total 1740000 (6 unidades, 3 productos).

Antes de la corrección (commit e4667f3):

```
[OK ] Listar sin productos
[OK ] Registrar P001
[FALLA] Registrar P001 nuevamente
        el programa terminó con error (código 1)
[OK ] Nombre vacío
[OK ] Nombre solo espacios
[OK ] Categoría fuera de la tupla
[OK ] Precio negativo
[OK ] Cantidad negativa
[OK ] Cantidad cero aceptada
[OK ] Buscar P001
[OK ] Buscar código desconocido
[OK ] Actualizar a 10 unidades
[OK ] Actualizar a -3 conserva cantidad
[OK ] Eliminar existente
[OK ] Eliminar inexistente no falla
[FALLA] Resumen enunciado (P001, P002)
        falta: 'P001 - Teclado - Periféricos - $85000.0 - 4 unidades'
        falta: 'P002 - Mouse - Periféricos - $45000.0 - 0 unidades'
        falta: 'Valor del inventario: $340000.0'
        falta: 'Categorías utilizadas: Periféricos'
[FALLA] Comprobación numérica (P001-P003)
        falta: 'Valor del inventario: $1740000.0'
        falta: 'Categorías utilizadas: Componentes, Periféricos'
[OK ] Opción no válida
[OK ] Salir

16/19 pruebas correctas
```

Después de la corrección (commit cedb766):

```
[OK ] Listar sin productos
[OK ] Registrar P001
[OK ] Registrar P001 nuevamente
[OK ] Nombre vacío
[OK ] Nombre solo espacios
[OK ] Categoría fuera de la tupla
[OK ] Precio negativo
[OK ] Cantidad negativa
[OK ] Cantidad cero aceptada
[OK ] Buscar P001
[OK ] Buscar código desconocido
```

```
[OK ] Actualizar a 10 unidades
[OK ] Actualizar a -3 conserva cantidad
[OK ] Eliminar existente
[OK ] Eliminar inexistente no falla
[OK ] Resumen enunciado (P001, P002)
[OK ] Comprobación numérica (P001-P003)
[OK ] Opción no válida
[OK ] Salir

19/19 pruebas correctas
```

Ejecución con el ejemplo del enunciado (listado, sin stock y resumen):

```
       GESTOR DE INVENTARIO
Opción: Código: Nombre: Categoría: Precio: Cantidad: Producto registrado.
       GESTOR DE INVENTARIO
Opción: Código: Nombre: Categoría: Precio: Cantidad: Producto registrado.
       GESTOR DE INVENTARIO
Opción:
LISTA DE PRODUCTOS
P001 - Teclado - Periféricos - $85000.0 - 4 unidades
P002 - Mouse - Periféricos - $45000.0 - 0 unidades
       GESTOR DE INVENTARIO
Opción: Sin existencias:
- Mouse
       GESTOR DE INVENTARIO
Opción:
RESUMEN DEL INVENTARIO
Productos: 2
Unidades: 4
Valor del inventario: $340000.0
Categorías utilizadas: Periféricos
       GESTOR DE INVENTARIO
Opción: Programa finalizado.
```

Reto de ampliación (movimientos, orden, diccionario, categoría y mayor valor):

```
Opción: Código: Nombre: Categoría: Precio: Cantidad: Producto registrado.
Opción: Código: Nombre: Categoría: Precio: Cantidad: Producto registrado.
Opción: Código: Nombre: Categoría: Precio: Cantidad: Producto registrado.
Opción: Código del producto: Movimiento (+ aumenta, - disminuye): Error: las existencias no pueden quedar
    negativas.
Opción: Código del producto: Movimiento (+ aumenta, - disminuye): Existencias actuales: 3
Opción: Disco SSD - P003
Mouse - P002
Teclado - P001
Opción: {'P001': 'Teclado', 'P003': 'Disco SSD', 'P002': 'Mouse'}
Opción: Categoría: Productos: Teclado, Mouse
Opción: Mayor valor: Disco SSD ($1400000.0)
Opción: Programa finalizado.
```

Reto de depuración:

```
Total incorrecto (sin cantidad): 830000.0
Total correcto (precio * cantidad): 1740000.0
Cantidad del primer producto: 4
```

6. Preguntas de análisis situacional

Pregunta 1

¿Qué harías si el gestor de inventario permite registrar dos productos con el mismo código?

Qué haría. Recorrería la lista productos con un for antes de agregar el nuevo diccionario, comparando el código ingresado con producto["codigo"] de cada elemento.

Por qué. El código identifica de forma única a cada producto. Si se repite, la búsqueda, la actualización y la eliminación afectarían solo al primero que aparezca y el inventario quedaría inconsistente.

Cómo lo implementaría. Normalizaría el código con strip().upper() para que "p001 " y "P001" cuenten como el mismo. Guardaría el resultado del recorrido en una variable (encontrado) y usaría break al hallarlo. Si encontrado no es None, mostraría el error y no ejecutaría el append; solo en el else se pediría el precio y la cantidad y se agregaría el producto. Otra opción es un conjunto de códigos ya usados, pero recorrer la lista evita mantener dos estructuras sincronizadas.

```
codigo = input("Código: ").strip().upper()
encontrado = None
for producto in productos:
    if producto["codigo"] == codigo:
        encontrado = producto
        break

if encontrado is not None:
    print("Error: ya existe un producto con ese código.")
else:
    ...  # pedir precio y cantidad, validar y hacer append
```

Cómo comprobaría el resultado. Registraría P001 dos veces y comprobaría que aparece el mensaje de rechazo y que al listar solo existe un P001. Está automatizado en la prueba "Registrar P001 nuevamente", que pasa.

Pregunta 2

¿Qué harías si al seleccionar una opción del menú el programa vuelve a mostrar las opciones, pero elimina todos los productos registrados anteriormente?

Qué haría. Revisaría dónde se inicializa la lista productos y moverla fuera del ciclo del menú.

Por qué. Si productos = [] está dentro del while, cada vuelta crea una lista vacía y descarta lo que había. Las colecciones que deben conservar información durante la ejecución se crean una sola vez, antes del ciclo; dentro del ciclo solo se modifican.

Cómo lo implementaría. Organizaría el programa en este orden: (1) constantes (CATEGORIAS), (2) colecciones (productos = []) y la variable opcion, (3) el while con el menú, donde únicamente se usan append, remove o se cambian campos. Las variables temporales (codigo, nombre, encontrado) sí pueden crearse en cada vuelta porque no guardan datos permanentes.

```
CATEGORIAS = ("Periféricos", "Componentes", "Accesorios")
productos = []        # una sola vez, ANTES del ciclo
opcion = ""

while opcion != "0":
    # productos = []  <- error: aquí se vaciaría en cada vuelta
    ...
```

Cómo comprobaría el resultado. Registraría un producto, elegiría otra opción (por ejemplo la 9, no válida) y luego listaría: el producto debe seguir ahí. Las pruebas automáticas hacen varias operaciones seguidas en una misma ejecución (registrar, listar, buscar, actualizar) y pasan. El caso fue provocado y explicado como error 1 de gestor_depuracion.py.

Pregunta 3

¿Qué harías si un usuario intenta actualizar las existencias de un producto utilizando una cantidad negativa?

Qué haría. Validaría el dato antes de tocar el diccionario y, si es negativo, mostraría el error sin modificar nada.

Por qué. Una cantidad negativa no tiene sentido para unas existencias y dañaría el resumen (unidades y valor). Si se asignara primero y se validara después, el dato incorrecto ya habría reemplazado al correcto.

Cómo lo implementaría. Buscaría el producto; con la nueva cantidad aplicaría if nueva_cantidad < 0: mostrar el error y el mensaje "Se conserva la cantidad anterior". Solo en el else asignaría encontrado["cantidad"] = nueva_cantidad. Se usa < 0 (no <= 0) para que el cero siga siendo válido: significa producto agotado.

```
nueva_cantidad = int(input("Nueva cantidad: "))

if nueva_cantidad < 0:
    print("Error: la cantidad no puede ser negativa.")
    print("Se conserva la cantidad anterior.")
else:
    encontrado["cantidad"] = nueva_cantidad
    print("Existencias actualizadas.")
```

Cómo comprobaría el resultado. Registraría P001 con 4 unidades, intentaría actualizar a -3 y luego buscaría P001: debe seguir mostrando Cantidad: 4. Después actualizaría a 10 y comprobaría que solo cambia la cantidad. Ambas pruebas están automatizadas y pasan.

Pregunta 4

¿Qué harías si necesitas obtener únicamente las categorías diferentes de los productos registrados, sin mostrar categorías repetidas?

Qué haría. Usaría un conjunto (set), construido con una comprensión de conjunto sobre la lista de productos.

Por qué. Un conjunto guarda cada valor una sola vez, de modo que los duplicados desaparecen sin escribir código. Una lista obligaría a comprobar manualmente si la categoría ya está antes de agregarla; una tupla es inmutable y no elimina repetidos; un diccionario exigiría inventar un valor para cada clave.

Cómo lo implementaría. Escribiría {producto["categoria"] for producto in productos}. Como un conjunto no garantiza el orden al imprimirse, para mostrarlo lo ordenaría con sorted() y lo uniría con ", ".join(...). El resultado se calcula a partir de los datos actuales, sin modificar los registros.

```
categorias_utilizadas = {producto["categoria"] for producto in productos}
print("Categorías utilizadas:", ", ".join(sorted(categorias_utilizadas)))
```

Cómo comprobaría el resultado. Registraría P001 y P002 (ambos Periféricos) y P003 (Componentes): el resumen debe mostrar 2 categorías, "Componentes, Periféricos", sin repetir Periféricos. Está cubierto por la prueba de comprobación numérica.

Pregunta 5

¿Qué harías si el resumen del inventario muestra un valor económico incorrecto, aunque los precios y las cantidades de cada producto son correctos?

Qué haría. Revisaría la fórmula del valor total y los datos de entrada con una prueba de resultado conocido.

Por qué. Si cada precio y cantidad es correcto, el error está en cómo se combinan: sumar solo precios sin multiplicar por cantidades, multiplicar por la variable equivocada, reiniciar el acumulador dentro del ciclo o escribir mal los datos (en este laboratorio, escribir 85.000 hizo que Python leyera 85.0 y el resumen mostró $1740.0).

Cómo lo implementaría. Calcularía a mano con P001-P003: 340000 + 0 + 1400000 = 1740000. Compararía con el resultado del programa e imprimiría temporalmente los subtotales por producto. El cálculo correcto es la suma de precio * cantidad de cada producto, con el acumulador iniciado en 0 antes del ciclo (o sum() con una comprensión). Si la fórmula es correcta, revisaría cómo se escribieron los datos.

```
valor_total = sum(p["precio"] for p in productos)                # incorrecto: 830000.0
valor_total = sum(p["precio"] * p["cantidad"] for p in productos)  # correcto: 1740000.0
```

```
# git commit -m "fix(2.9): calcular el valor del inventario como precio * cantidad"
```

Cómo comprobaría el resultado. Con la prueba "Comprobación numérica" el resultado debe ser $1740000.0. El error provocado en gestor_depuracion.py daba 830000.0 (solo suma precios) frente al correcto 1740000.0. Para documentar en GitHub: usaría git log y git diff para ver qué cambió, corregiría, ejecutaría python pruebas_gestor.py (19 de 19) y haría un commit como "fix(2.9): calcular el valor del inventario como precio * cantidad", explicando la causa y el resultado.

7. Conclusión

El repositorio conserva los laboratorios 01 y 02. El gestor de inventario cumple las reglas del enunciado, pasa las 19 pruebas automáticas y el historial de commits permite verificar cada cambio descrito en este informe.